Zephyr Project

IT-Schulungen.com
Technology & Transformation

5.02 Zephyr-Bootprozess und Startsequenz

Resetvektor · Kernelinitialisierung · Anwendungseinstieg

5.02

Der Bootpfad verbindet Hardware und Anwendung

RESET

Architektur übernimmt

Resetvektor, Anfangsstack und hardwarenahe Vorbereitung

KERNEL

Zephyr initialisiert

Speicher, Kernelobjekte, Geräte und Subsysteme

ANWENDUNG

main() startet

Der Main-Thread übernimmt nach abgeschlossener Initialisierung

Jede Startphase schafft definierte Voraussetzungen für die nächste.

Speicher vor dem Kernelstart vorbereiten

01	`Stack`	Ein Anfangsstack ermöglicht die ersten C-Funktionsaufrufe.
02	`.data`	Initialisierte Daten werden bei XIP von ROM nach RAM kopiert.
03	`.bss`	Nicht initialisierte globale Daten werden auf null gesetzt.

Linkerskript und Startup-Code müssen dieselbe Speicherbelegung voraussetzen.

Die Startsequenz in fünf Schritten

Vom CPU-Reset bis zur ausführenden Anwendung

01

Resetvektor

definierter Einstieg

02

C-Laufzeit

Stack, .data und .bss

03

z_cstart()

frühe Kernelinitialisierung

04

Init-Level

System und Geräte ordnen

05

main()

Anwendung im Main-Thread

Der konkrete Architektur-Startup-Code variiert, die gemeinsamen Phasen bleiben.

Init-Level ordnen den Systemstart

REIHENFOLGE

```
EARLY
PRE_KERNEL_1
PRE_KERNEL_2
POST_KERNEL
APPLICATION
```

01 Vor dem Kernel

EARLY und PRE_KERNEL laufen ohne allgemein verfügbare Kernelprimitiven.

02 Mit aktivem Kernel

POST_KERNEL darf Kernelprimitiven nutzen; APPLICATION liegt direkt vor main().

Prioritäten ordnen Einträge innerhalb desselben Initialisierungslevels.

SYS_INIT(): Level und Priorität

BEISPIEL

```
SYS_INIT(init_fn, POST_KERNEL, 60);
```

LEVEL	bestimmt die verfügbare Laufzeitumgebung
PRIORITÄT	ordnet Einträge innerhalb eines Levels von 0 bis 999
ABHÄNGIGKEIT	muss über Level, Priorität oder Gerätebeziehung abgebildet sein
KONTEXT	entscheidet, welche Kernel- und Treiberdienste bereits nutzbar sind

Ein früher Init-Zeitpunkt ist nur sinnvoll, wenn seine Voraussetzungen erfüllt sind.

Wer startet wann?

BOOTPFAD

```
RESET → z_cstart() → Init-Level → main()
```

EARLY	direkt nach Eintritt in die C-Laufzeit
PRE_KERNEL_1/2	Kernelinitialisierung ohne verfügbare Kernelprimitiven
POST_KERNEL	Kernel lebt; Kernelprimitiven sind verfügbar
APPLICATION	letzte Systeminitialisierung unmittelbar vor der Anwendung
C++-Konstruktoren	nach der Treiberinitialisierung und vor main()

main() läuft im vom Kernel erzeugten Main-Thread.

Technische Kernaussagen

STARTUP

Hardware-nah

Resetvektor

Anfangsstack

Speicheraufbereitung

INIT

Geordnet

feste Level

Priorität je Level

klare Voraussetzungen

ANWENDUNG

Thread-basiert

```
Kernel initialisiert
C++-Konstruktoren
ausgeführt
main() aufgerufen
```

Bootcode, Linkerskript und Initialisierungsordnung bilden gemeinsam den Startvertrag.